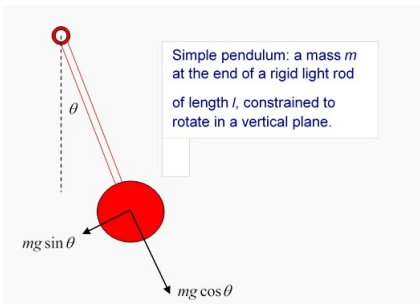
Tutorial Three: Oscillators

Simple Pendulum and Damped Harmonic oscillator

My friend has a new job to catch lobsters. Whenever he was asked what he was doing, he said he had a

1. Pendulum with small angles

Image taken from Michael Fowler



We are all very familiar with the simple pendulum above. The differential equation of the pendulum is given by

$$\ddot{\theta} + \frac{g}{l}\sin\theta = 0 \tag{1}$$

In the small angle approximation $\sin\theta \sim \theta$, the equation simplifies to

$$\ddot{\theta} + \frac{g}{l}\theta = 0 \tag{2}$$

with a known solution;

$$\theta(t) = \theta_0 \cos\left(\sqrt{\frac{g}{l}}t\right). \tag{3}$$

In the first part of the tutorial, we will use neural networks to recover the value of gravity g. The input consists of the angles and the corresponding time. We will assume that we measured the length of the pendulum l and the initial angle θ_0.

In the first part of the notebook, we generated some training data according to the analytical solution and took some random samples of it as input to the neural network. Then we define a standard neural network.

Our neural network (NN) has one objective: trying to find $g = \omega^2 l$ in the analytical solution. We ask the NN to output ω and use $g = \omega^2 l$ to compute g.

The first training epoch will produce nothing but a random list of numbers as an initial frequency.

a) Frequency is one number, not a list. Write one line code to use the average frequency as the output of the network.

   Use the analytical solution of SHO, we can easily (implemented in the code) predict θ_{NN} of the neural network with the average frequency.

   The loss function is the key to train a neural network. Now it is time to construct it. It contains two parts.

b) First write code to penalize deviation from the mean, after all, it is suppose to be one number.

c) Second, write code to compute the mean of the square errors.

   Now train the network for 20 epochs.

d) Examine the error and the lost function as a function of time in the next block. Should we train more epochs? Keep track of the loss, when should we stop training?

e) Extrapolate the network prediction to a longer period of time. How does the network perform?

f) There are many things that you can explore (but do it at the end of the tutorial)

   - Compare the network prediction with the data at some random number of epochs. Does the deviation between the prediction and the data make sense according to your cost function plot?

- Explore: change the activation function from `nn.Tanh()` to `nn.Sigmoid()` or `nn.ReLU`. What is your discovery?
- Explore the learning rate, what happens if we change the learning rate to 0.01 or 0.0001? Try different combinations of activation function and learning rate.
- Explore the optimizer: what if you use SGD or RMSprop?
- Explore the hidden layers: what if there are more hidden layers in the network?
- Now we are going to add some noise to our training data, since we cannot measure the angles so precisely after all. Redo your analysis, how does our NN perform?

2. PINN for damped oscillators

The damped oscillator model is

$$x''(t) + 2\gamma\, x'(t) + \omega^2 x(t) = 0$$

We will use `solve_ivp` to generate data between the time interval $t \in [0, 20]$ using the following parameters $\gamma = 0.1$, $\omega = 1$ and the initial conditions $x(0) = 1$, $x'(0) = 0$.

a) Write a code to define our first-order system (only `dvdt` part is needed)

b) Write a code to select your training data (We may change later). *Tip: You don't want to feed all your data as training data to the network, leave some for prediction.*

c) Run the block to visualize the solution to make sure that it is expected (as a damped oscillator).

   First, let us see how a normal neural network will perform.

d) What is the discrepancy between the NN prediction and the ground truth? Can you explain why?

   In the following we will build a continuous PINN step by step.

a) We need to treat γ and ω as trainable. We need to modify the previous network by adding some parameters to the neural network. Examine the PINN network, find where the parameters are added as trainable.

b) The loss function now is a combination of three parts

$$\text{loss} = \text{loss}_{\text{data}} + \lambda_{\text{physics}}\text{loss}_{\text{phyics}} + \lambda_{\text{i.c.}}\text{loss}_{\text{i.c.}} \tag{4}$$

- The data Mean Square Error (MSE)(this is what we used for our normal NN). This is implemented.
- Choose a number of collocation points that you want to use to reinforce physics. Then compute the physics error from the ODE. You need to write code for this.
- Initial-condition penalty. You need to write code for this.

Note that you may want to use λ's to weigh the three components differently to improve efficiency during training. What happens if λ_{physics} is too big? What happens if λ_{physics} is too small?

e) Now we can train our PINN. Pick some number of epochs to start with and plot the loss every some number of epochs.

- You might find the convergence very slowly for our current problem. Examine the oscillator model and the parameters. What could be the problem? (*Hint: it is an* ODE) What should we do?
- Scaling: recall that machine learning algorithms like numbers from 0 to 1. If we do some scaling, the differential equation is going to change. Find out how it changes the ODE and change your code accordingly.
- Plot the total loss (although to keep track of what the model is doing, you probably also want to keep track of the three losses separately) and the convergence of γ, ω. How stable is your PINN? How far could your initial γ and ω be off so that the network can still learn the true values?

To see how our PINN performs, we plot the prediction against the ground truth.

f) Compare the predicted $x(t)$ and inferred parameters with ground truth; discuss discrepancies.

g) Some improvements/adjustments we can make

- Based on physics, we know what the kind of numbers γ and ω are. It is silly to train them to be negative. Instead, we can do the exponential trick to guarantee that we are only training positive numbers. Try to implement this.
- Play with learning rate, the weight (λ's) of physics and initial condition losses to see if there is any improvements.
- Play with different activation functions. Instead of `Tanh`, there are also `ReLU` or `Sigmoid`.
- How little data can we feed to PINN and it will still work? How robust is your model against noise?

A. Acknowledgments

The first part of the tutorial is based on this paper fresh in May 2025 section III A and the accompanying notebook.

The second part of the tutorial is based on this notebook.

new role, net work.